

InsureAI: End-to-End AI Assistant for Insurance

ML • DL • CNN • RNN • Generative AI • Prompt Engineering • LLM APIs

CAPSTONE PROJECT KICKOFF

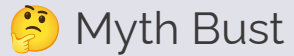


Welcome! Quick Icebreaker



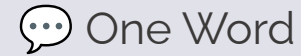
Have you ever filed an insurance claim?

Type Y / N in chat!



"AI will replace insurance agents."

True or False? Drop your vote!



Type **1 word** in chat that describes "insurance" to you right now!

What We're Building Today

01

Module 1 – ML

Premium Prediction + Fraud Detection

02

Module 2 – ANN

Deep Learning on tabular data

03

Module 3 – CNN

Vehicle damage image classifier

04

Module 4 – LSTM

Customer sentiment analysis

05

Module 5 – GenAI

LLM-powered insurance chatbot



The Problem: Manual Insurance Chaos



Slow Claims

Days or weeks to settle



Fraud Losses

\$80B lost annually in the US



Poor CX

Customers left waiting, frustrated



High Costs

Massive operational overhead

InsureAI — The Solution

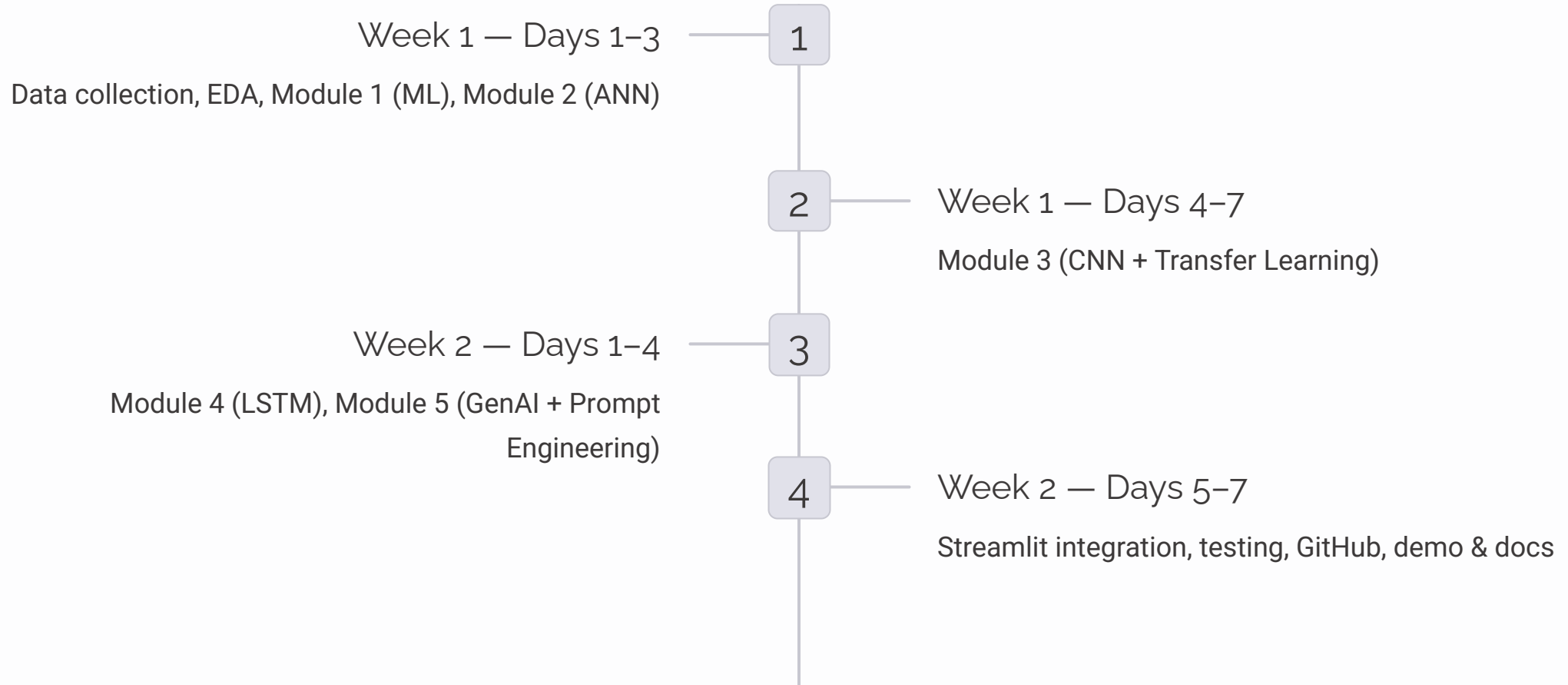


One App. Five AI Modules.

- Instant premium quotes for customers
- Auto-flag fraudulent claims
- Photo-based damage verification
- Sentiment tracking from reviews
- 24x7 AI chatbot for policy Q&A

☐ All modules live inside **one Streamlit app** — your capstone deliverable!

Your Project Roadmap





Before We Code — Datasets First!

1

insurance.csv

Medical Cost — ~1,300 rows — Kaggle

2

insurance_claims.csv

Fraud Detection — ~1,000 rows — Kaggle

3

Car Images

Damaged vs Whole — ~2,000 images — Kaggle

4

Review Text

Customer Feedback + Ratings — Kaggle

5

FAQ Text

10–15 Q&A pairs — **you create this!**



Chat check: Have you downloaded your datasets yet? Type  or  in chat!



Project Folder Structure

InsureAI/

```
|—— data/
|   |—— insurance.csv
|   |—— insurance_claims.csv
|—— models/
|   |—— premium_model.pkl
|   |—— fraud_model.pkl
|—— notebooks/
|   |—— module1_ml.ipynb
|   |—— module2_ann.ipynb
|—— app/
|   |—— streamlit_app.py
|—— .env
|—— .gitignore
|—— requirements.txt
```

Golden Rules 🏆

- One notebook per module
- Never commit your `.env` file
- Commit after each module milestone
- Meaningful variable names + comments



NEVER hard-code your API key. Use `python-dotenv` always!

Step 1: Data Understanding & EDA

Before modeling, **know your data**. Run these 5 commands first:

```
df.head()
```

See first 5 rows

```
df.info()
```

Column types & nulls

```
df.describe()
```

Stats summary

```
df.isnull()
```

Missing values check

```
df['col'].value_counts()
```

Category distribution

Micro Task #1 — 2 Minutes!

Your Task

Open `insurance.csv` in a Jupyter notebook. Run `df.head()` and `df.describe()`.

What's the average age? Drop it in chat! 🎯

Hint

Expected columns: `age, sex, bmi, children, smoker, region, charges`

Target = `charges`



You have exactly 2 minutes. Fingers on keyboards — GO!

Insurance Dataset — What Each Column Means



age / sex / children

Policyholder demographics — key risk indicators



bmi

Body Mass Index — higher BMI = higher health risk



smoker

Most powerful predictor — smokers pay ~3× more



region

Geographic zone — affects local healthcare costs



charges

Annual medical cost — our **regression target** 🎯



EDA Visuals You Must Include

Histogram

Distribution of charges —
right-skewed?

Box Plot

smoker vs charges —
huge difference!

Heatmap

Correlation matrix —
which features matter
most?

Scatter

age vs charges colored by
smoker

Key EDA Insight 💡

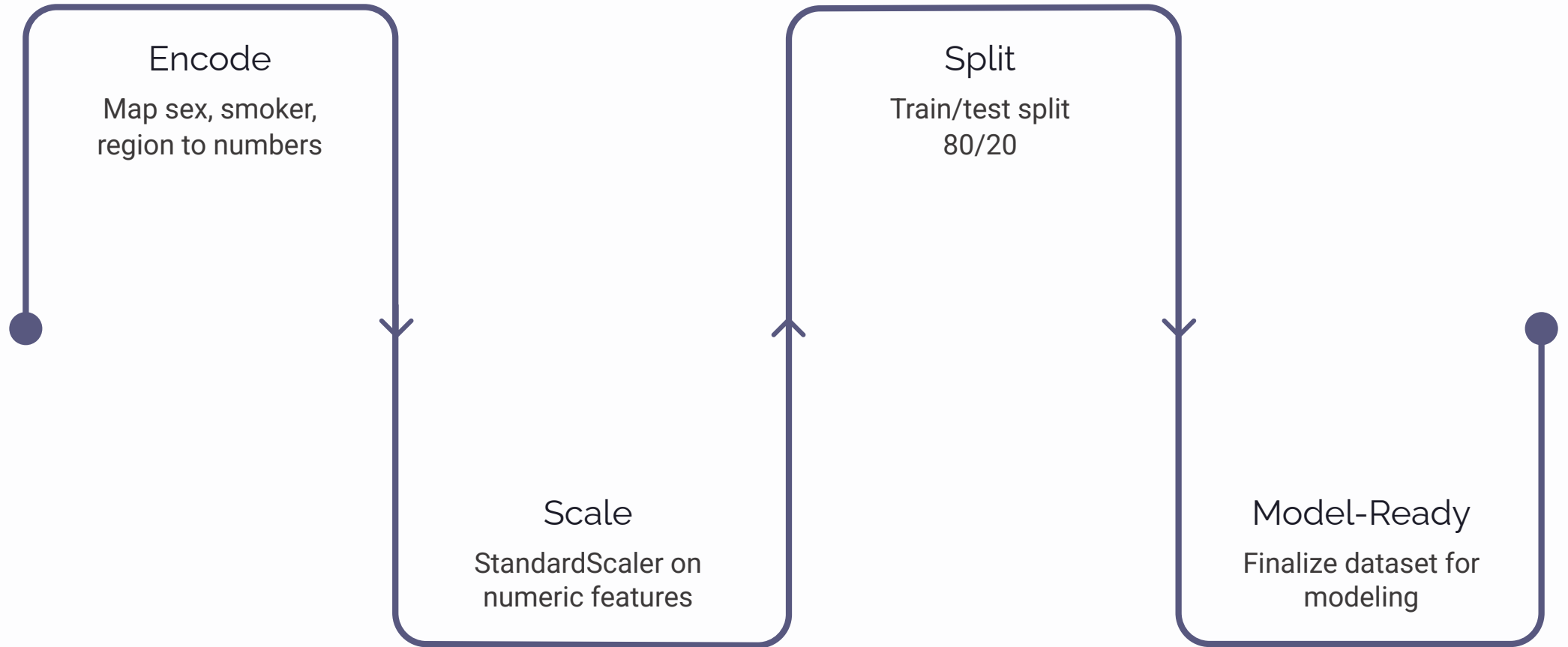
Smokers' charges cluster around **\$30k–\$60k** while non-smokers average **\$8k–\$10k**.

This single feature can make or break your model!



Quick Poll: Which feature do you think is #1 predictor? Type it in chat!

Preprocessing — Making Data Model-Ready



Encode → Scale → Split → Model. Always in this order. Never shuffle steps!

Encoding Categoricals — Quick Guide

Label Encoding

Use for **binary** columns

sex: male=1, female=0
smoker: yes=1, no=0

One-Hot Encoding

Use for **multi-class** columns

region → NE, NW, SE, SW
(4 dummy columns)

Why Does This Matter? 🤔

ML models speak **numbers only**. Text columns confuse them!

- Use `pd.get_dummies()` or `LabelEncoder`
- Drop one dummy column to avoid multicollinearity
- Apply the **same** encoder at prediction time in Streamlit

CHECKPOINT #1

Take a breath. You've covered: Data Setup, EDA, and Preprocessing. What's ONE keyword from this section? Drop it in chat! 

 Data Loaded

 EDA Done

 Encoded

 Scaled

 Train Models

MODULE 1 — PART A

Premium Prediction — Regression

Business Goal: Given a new customer's profile, predict their annual insurance premium *instantly* — no manual underwriting needed.



3 Models to Train — Regression

1

Linear Regression

Your baseline. Simple, fast, interpretable. Start here.

2

Random Forest Regressor

Ensemble of decision trees. Handles non-linearity well.

3

XGBoost / Gradient Boosting

Top performer on tabular data. Aim for best R^2 .



Target: $R^2 \geq 0.75$ on test set. If you hit 0.85+, you're crushing it!

Regression Metrics — Plain English



R^2 Score

"How much variance does my model explain?" 1.0 = perfect.
 ≥ 0.75 = good.



MAE

Mean Absolute Error — average dollar difference from actual.
Lower is better.



RMSE

Punishes big errors more than MAE. Use to catch large prediction mistakes.

This or That — Quick Vote!

A new customer aged 45, BMI 32, smoker, 2 kids — what do YOU predict?

Option A

~\$8,000 annual premium

Option B

~\$28,000 annual premium

Option C

~\$45,000 annual premium

 Type A, B, or C in chat — no wrong answers, just your gut feeling! 

Module 1 — Part B: Fraud Detection



The Problem

Only ~6% of **claims are fraudulent** — but each costs thousands of dollars.

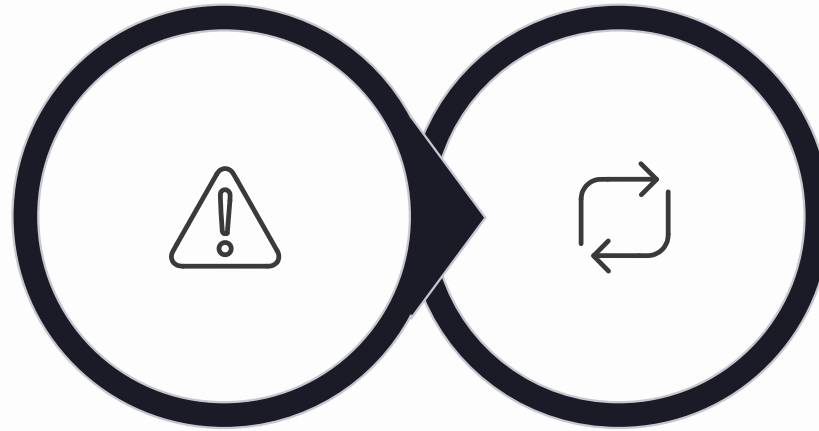
Your Target

fraud_reported: Y / N

The Challenge

Severe **class imbalance** — 94% legitimate, 6% fraud. A naive model says "not fraud" always and gets 94% accuracy — but catches 0 frauds! 🤖

Handling Class Imbalance — SMOTE



Problem

SMOTE Fix

SMOTE = Synthetic Minority Over-sampling Technique. It *creates new synthetic fraud examples* to balance training data. Use `imblearn.over_sampling.SMOTE`.

Fraud Detection — Metrics That Matter

Precision

Of flagged frauds, how many were actually fraudulent? Avoid false alarms.

Recall

Of all real frauds, how many did we catch? Missing fraud = money lost!

F1-Score

Balance of Precision + Recall. **Target ≥ 0.70** on minority (fraud) class.

ROC-AUC

Overall discriminating power of the model. Higher = better separation.



In fraud detection, **Recall** matters more than Accuracy. A missed fraud is costlier than a false alarm!



Saving Your Best Models

```
import joblib

# After training best model:
joblib.dump(best_reg_model,
            'models/premium_model.pkl')

joblib.dump(best_clf_model,
            'models/fraud_model.pkl')

# To load in Streamlit:
model = joblib.load(
    'models/premium_model.pkl')
prediction = model.predict(X_new)
```

Why Save Models?

Training takes time. In Streamlit, you **load** the saved model and predict instantly — no retraining needed!

- Use `joblib` for sklearn models
- Use `.h5` or `.keras` for deep learning
- Always save the **scaler** too!



Module 1 Mini Quiz

Q1 — MCQ

Which metric best measures fraud detection quality when classes are imbalanced?

A) Accuracy B) F1-Score C) R^2 D) MAE

Q2 — Scenario

Your model gets 93% accuracy on fraud data, but $F1 = 0.12$. Is this a good model? Why?

Q3 — Trick ?

"More training data always improves the model." True or False? Explain!



Answer in chat — most creative answer gets a virtual 🏆!

😄 Energy Reset — Fun Fact!

"The first insurance policy ever written was in 1347 — for a ship carrying cargo from Genoa to Majorca. No AI was involved... yet!"



😂 Fun Reality

Insurance agents in 1347 used gut instinct. You're using XGBoost. Who wins?

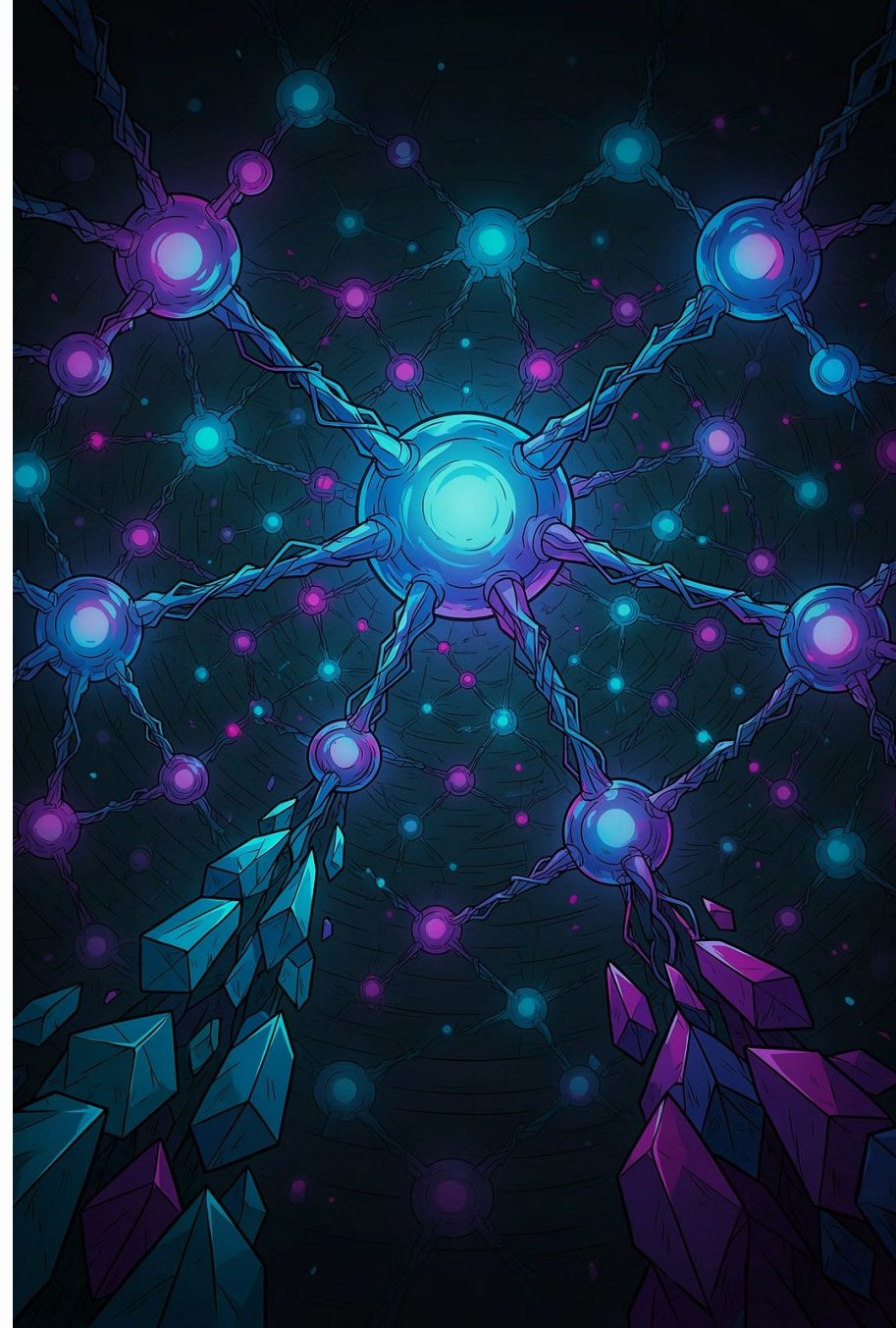
🧘 Micro Break

Relax your shoulders. Take one deep breath. We're halfway through Module 1!

MODULE 2 — DEEP LEARNING

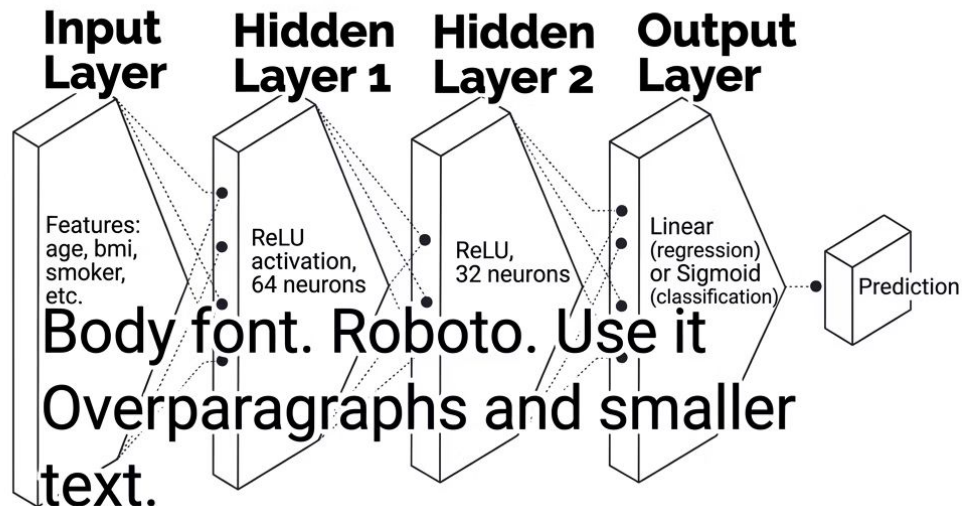
ANN — Artificial Neural Networks

Same tabular data. Deeper intelligence. Let's go beyond classical ML!



ANN Architecture — Plain English

Raleway



ABCDEFGHIJKLMNOPQRSTUVWXYZ

This is the accent color, use it
for highlights and decorations
0123456789.

Key Choices

- **Hidden layers:** 2–3 is enough for tabular data
- **Activation:** ReLU for hidden, Linear/Sigmoid for output
- **Optimizer:** Adam (default, works great)
- **Loss:** MSE for regression, Binary Crossentropy for classification

Preventing Overfitting in ANN



Dropout

Randomly "turns off" neurons during training. Prevents memorization. Use 0.2–0.3.



EarlyStopping

Stops training when validation loss stops improving. Saves time + prevents overfit.



Validation Split

Keep 20% of training data as validation. Watch train vs val loss curves — they should track together!

ANN vs ML — Comparison Table

Metric	Linear Reg	Random Forest	XGBoost	ANN
R ² Score	~0.75	~0.85	~0.87	~0.84
MAE (\$)	~4,200	~2,700	~2,500	~2,900
Training Speed	Very Fast	Fast	Fast	Slower
Interpretability	High	Medium	Medium	Low

📄 Include this table in your project report! It shows you evaluated rigorously. 📊

Micro Task #2 — ANN Debugging

"My ANN training loss keeps going down but validation loss goes UP after epoch 5. What's happening and how do I fix it?"

Your Answer?

Type 1–2 words in chat that describe the problem!

Fix It!

What would YOU do? Add Dropout? EarlyStopping? More data? Vote!



90 seconds. Type your diagnosis in chat — GO!

CHECKPOINT #2

Modules 1 & 2 done! ML + Deep Learning on tabular data = 

What keyword from Module 2 surprised you most? Drop it in chat! 

→ Regression → Premium Prediction

→ Classification → Fraud Detection

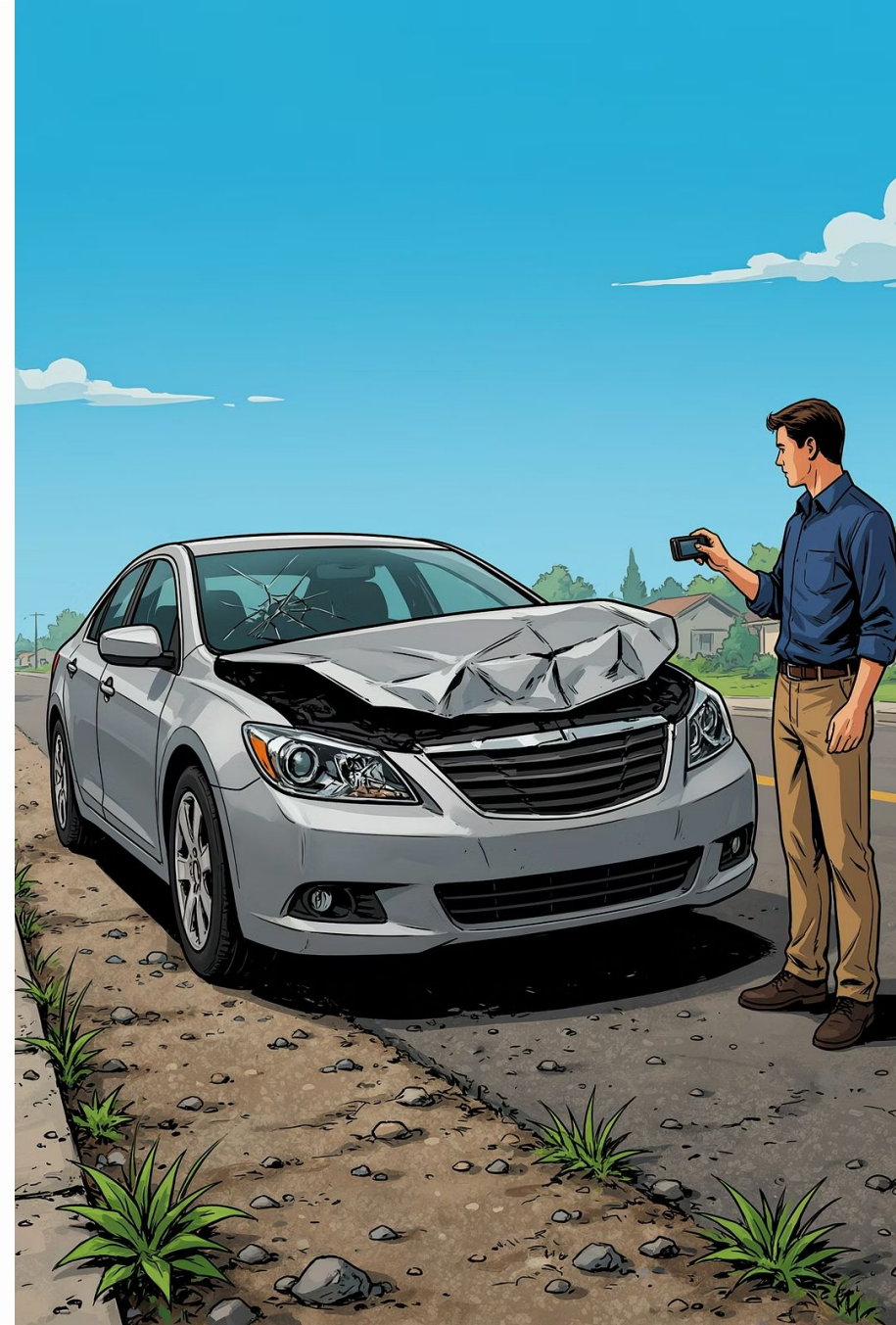
→ ANN → Deep Learning comparison

→ Models saved → Ready for Streamlit

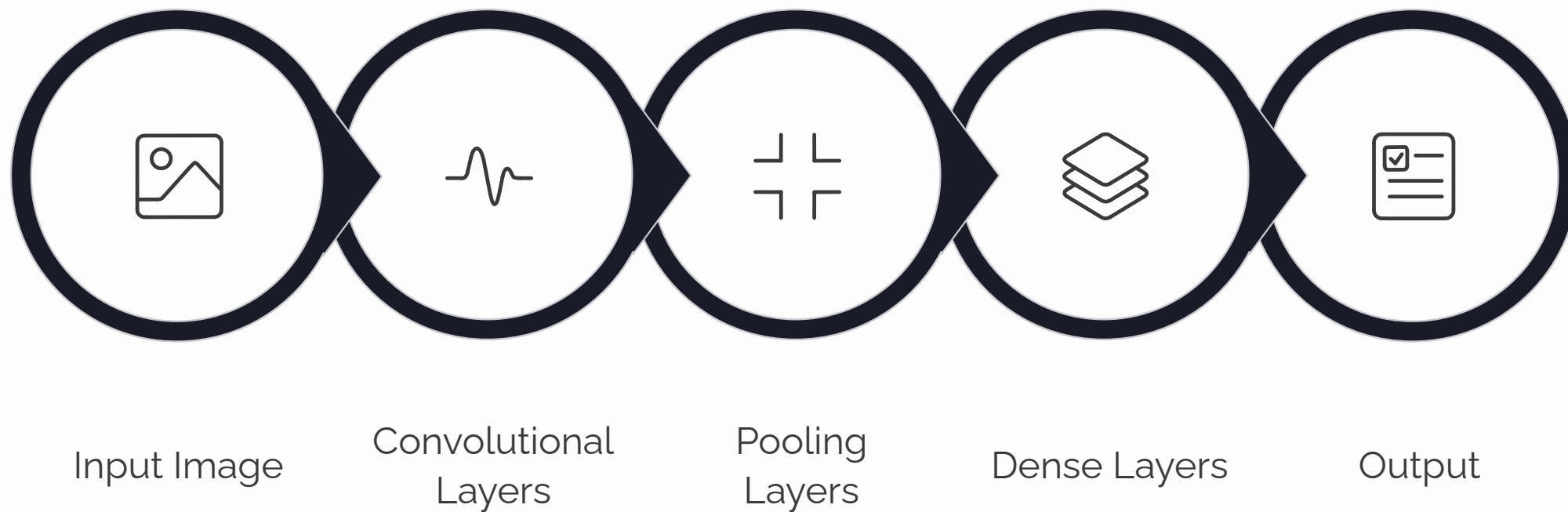
MODULE 3 — CNN

Vehicle Damage Detection with CNN

Customer uploads a photo. InsureAI says: **Damaged** ✓ or **Intact** ✗ — in seconds!

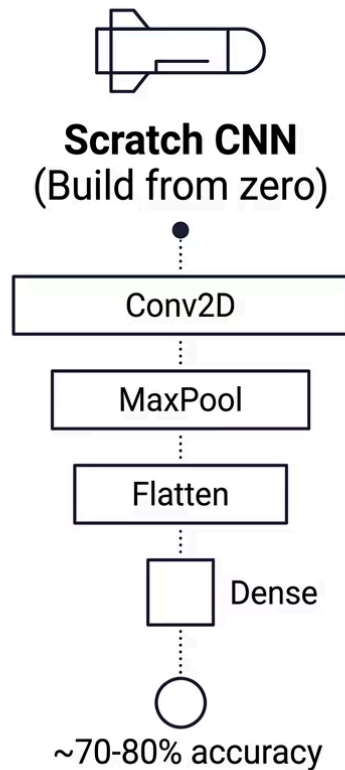


How a CNN "Sees" an Image

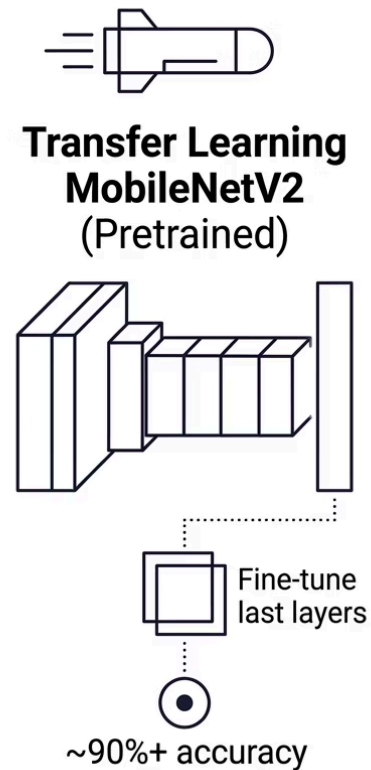


Convolution detects features like edges and dents. Pooling compresses. Dense layers decide. Think of it like a checklist of visual clues.

Two Approaches — Scratch vs Transfer Learning



More training time.
Needs more data.



Fast.
Needs less data.

Recommendation 💡

Always try **Transfer Learning first** for image tasks with small datasets.

- MobileNetV2 — lightweight, fast
- VGG16 — heavier but powerful
- Freeze base layers, train only the top

📘 Target: **≥85% test accuracy**. Transfer learning makes this achievable even with ~2,000 images!

Image Preprocessing Checklist

01

Resize

All images → 224×224 pixels. Consistency is key!

02

Normalize

Pixel values 0–255 → 0–1. Divide by 255.0.

03

Augmentation

Rotation, horizontal flip, zoom — creates variety from limited images.

04

Split

80% train / 20% validation. Use `ImageDataGenerator`.



Hypothetical: What Would YOU Do?

"A customer uploads a blurry, low-resolution photo of their car. Your CNN predicts 'Not Damaged' with 51% confidence. What do you do?"

Option A

Accept the prediction and auto-approve the claim

Option B

Flag for manual review if confidence < 70%

Option C

Ask customer to re-upload a clearer photo

📝 Type A, B, or C — then tell us WHY! Best reasoning wins bragging rights. 🏆

Transfer Learning Code — Key Steps

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model

# Load pretrained base (freeze weights)
base = MobileNetV2(weights='imagenet',
    include_top=False,
    input_shape=(224,224,3))
base.trainable = False

# Add custom classification head
x = GlobalAveragePooling2D()(base.output)
x = Dense(64, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base.input, outputs=output)
model.compile(optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
```

CNN Case Study — Real World Impact



Before InsureAI CNN

Claims adjuster drives to the site. 3–5 business days. High cost per visit.

After InsureAI CNN

Customer uploads photo via app. CNN verifies damage in **<2 seconds**. Adjuster only involved for edge cases.

Business Impact 💰

60% reduction in physical inspection costs.
Claims settled in hours, not days.

Fun Check-In!

Drop an emoji that describes how you're feeling right now:

 Crushing it  Still processing  Super energized  Need coffee

 No wrong answers. This is purely to confirm you're still with us! 

MODULE 4 – RNN/LSTM

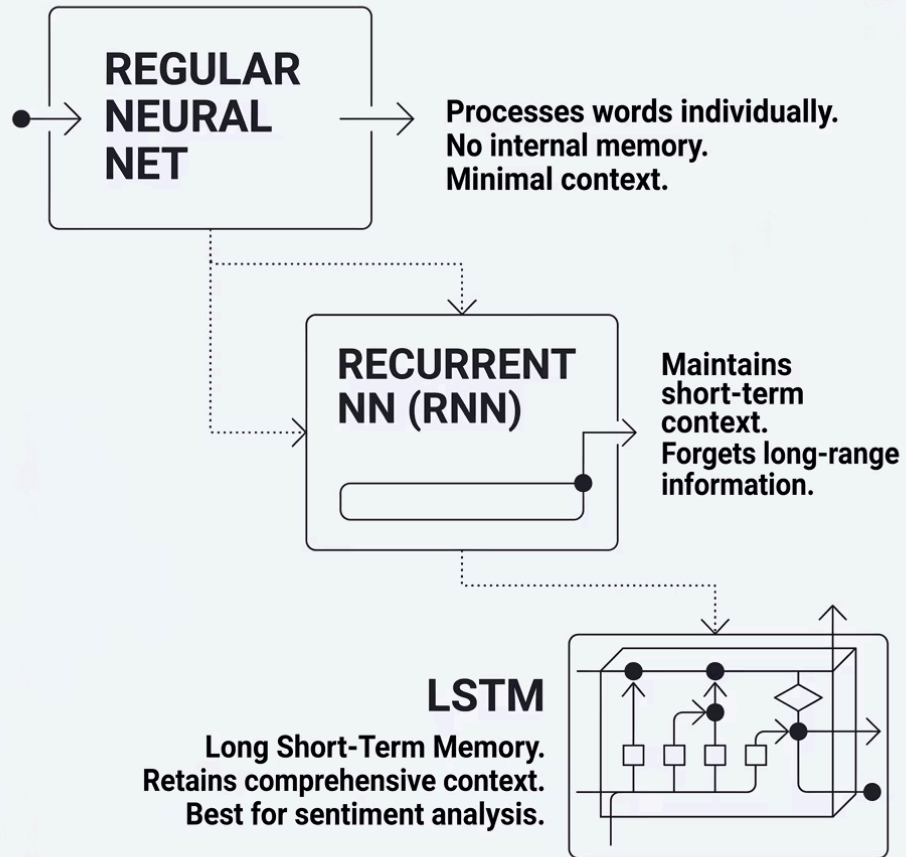
Customer Sentiment Analysis with LSTM

"My claim was settled very fast!" → **Positive** ✓

"Waited 3 months with no response." → **Negative** ✗



Why LSTM for Text?



The Key Problem

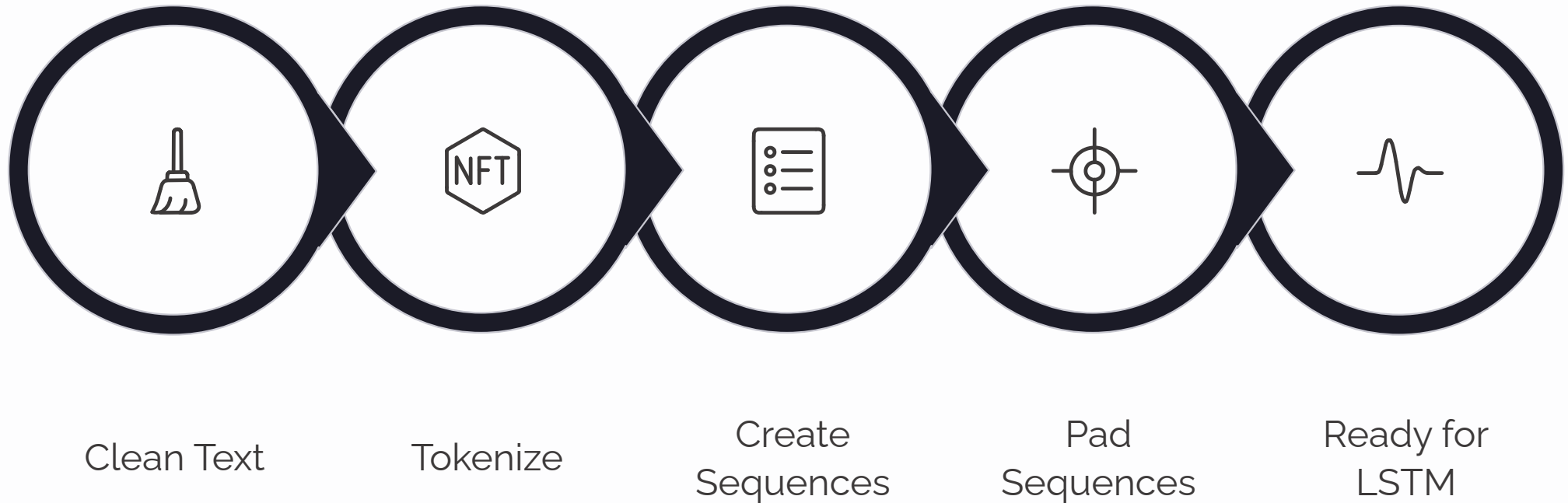
Words depend on **context**. "Not bad" is positive. "Not good" is negative. Regular models miss this!

LSTM Solution

LSTM remembers word order and context across the entire review using **memory cells and gates**.

i Target: **≥85% accuracy** on customer review sentiment classification.

Text Preprocessing Pipeline



Every sentence must become a **fixed-length number sequence** before LSTM can process it.

LSTM Model Architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Embedding, LSTM, Dense, Dropout)

model = Sequential([
    # Embedding: word → dense vector
    Embedding(vocab_size, 64,
    input_length=max_len),
    # LSTM: sequence memory
    LSTM(64, return_sequences=False),
    Dropout(0.3),
    # Output: sigmoid for binary
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
```



Micro Task #3 — Sentiment Prediction!

Which of these reviews is Positive vs Negative? Classify manually first, then let your model confirm!

Review A

"The insurance team was responsive and my claim was approved within 24 hours. Absolutely amazing service!"

Review B

"I have been waiting for 6 weeks for any response. Not a single callback. Totally disappointed."

Review C

"Service was okay but the claim process could be much simpler. Not bad, not great."

📋 Type A=P/N, B=P/N, C=P/N in chat! Review C is the tricky one... 🤔

CHECKPOINT #3

CNN + LSTM done! Halfway through the technical deep-dive.

Rate your confidence 1–5 in chat: 1=Lost 😓 → 5=Expert 🚀

✓ CNN

Damage detection — images
classified

✓ LSTM

Sentiment classified — reviews scored

 GenAI

The most exciting module coming up!

MODULE 5 — GENERATIVE AI

InsureAI Chatbot — Powered by LLM

Policy Q&A • Claim Summariser • Email Drafter

No dataset needed — just great prompts and an API key!



Which LLM API Should You Use?



Groq

Fastest inference. Free tier generous.
Best for real-time chat. Start here!



Google Gemini

Free tier available. Strong multilingual
support. Google ecosystem friendly.



OpenAI

GPT-4o — industry gold standard.
Small credit (\$5) goes a long way for a
project.



Store API key in `.env` file. Use `python-dotenv`. Add `.env` to `.gitignore`. Non-negotiable!

Safe API Key Setup

.env file (NEVER commit this!)

```
GROQ_API_KEY=gsk_XXXXXXXXXXXX
```

In your Python code:

```
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

```
api_key = os.getenv("GROQ_API_KEY")
```

.gitignore file:

```
.env
```

```
__pycache__/
```

```
*.pkl
```

```
*.h5
```

Why This Matters

Leaked API keys get **scraped by bots within minutes**. You'll receive a surprise bill!

- Never put keys in notebooks
- Never push .env to GitHub
- Rotate keys if accidentally exposed

Prompt Engineering — 4 Techniques You Must Document

1

Zero-Shot

Just ask — no examples. "What documents do I need to file a claim?"

2

Few-Shot

Give 2–3 examples before the question. Guides the model's response format.

3

Role / Persona

"You are a senior insurance claims officer with 20 years experience..."

4

Chain-of-Thought

"Think step by step. First identify... then assess... finally conclude..."

Weak Prompt vs Engineered Prompt

Weak Prompt

"Answer insurance questions."

*Result: Vague. Answers anything. No persona. Goes off-topic.
No structure.*

Engineered Prompt

"You are a professional InsureAI assistant for ABC Insurance. Only answer insurance-related questions. If asked anything unrelated, politely redirect. Always reply in bullet points. Here are 3 examples: [few-shot examples]."

Result: On-topic, structured, professional, consistent.



Include before/after screenshots of these two prompts in your Prompt Engineering Report! 📸

3 GenAI Features — Prompts in Action



Policy Q&A Chatbot

System prompt restricts to insurance topics. Inject your 10–15 FAQ pairs as context. Persona: senior agent.



Claim Summariser

Takes long claim description. Returns: Incident | Date | Damage | Severity. Ask for JSON output format.



Email Drafter

Input: claim status + customer name.
Output: professional, empathetic claim status email. Chain-of-thought prompt.



Live Prompt Challenge!

"Write a system prompt for the InsureAI chatbot that: (1) Sets a professional persona, (2) Restricts to insurance topics only, (3) Uses a helpful, empathetic tone."



You Have

3 minutes. Write it in a text editor or notepad right now!



Share

Paste your first 2 lines in chat — best prompt gets featured in the session!

Module 5 — Mini Quiz

Q1 — MCQ

Which prompt technique gives the model 2–3 input-output examples to follow?

A) Zero-shot B) Few-shot C) Chain-of-thought D) Role prompting

Q2 — True/False

"It's fine to put API keys directly in Streamlit code as long as the repo is private." — True or False?

Q3 — Scenario

The chatbot keeps answering non-insurance questions about cooking recipes. Which prompt technique would fix this most effectively?

INTEGRATION

Building the Streamlit App

Five modules. One sidebar. Zero complexity for end users.



Streamlit App Structure

```
import streamlit as st
import joblib, os
from dotenv import load_dotenv
load_dotenv()

# Sidebar navigation
page = st.sidebar.selectbox(
    "Navigate", [
        "
```

Streamlit Premium Prediction Page

```
def run():  
    st.title(")
```

Micro Task #4 — Streamlit Scavenger Hunt!

Open the Streamlit docs (docs.streamlit.io) and find the widget to upload an image file. What is the function name?

Hint

You'll need this for the Damage Detection page (CNN module)!

Bonus

How do you display the uploaded image back to the user?
What's the function?

 Type both function names in chat — 2 minutes! 

Error Handling in Streamlit

Invalid Inputs

Use `st.error()` to show user-friendly messages. Check for empty fields before prediction.

API Failures

Wrap LLM calls in `try/except`. Show "Service temporarily unavailable" if API times out.

Missing Image

Check if `uploaded_file` is not `None` before processing.
Never assume file was uploaded.

Model Not Found

Check model file exists with `os.path.exists()` before loading.
Show clear error if missing.

CHECKPOINT #4 — Almost There!

All 5 modules built and integrated into Streamlit! Take a deep breath. This is your capstone. You've come so far! 🎉
Type 🔥 in chat if you're still with us!

✓ ML Models

✓ ANN

✓ CNN

✓ LSTM

✓ GenAI

✓ Streamlit

DEPLOYMENT & SUBMISSION

GitHub, Deployment & Final Delivery

Code that isn't on GitHub doesn't exist. Commit early. Commit often.

